

1 📦 Project Source & Setup

- **Base GitHub Repository:** Download the repository from [Aasuyadav1/PDF_RAG](#) (or [NitinPandit/RAG-Document-Nodejs](#)).
- **Target Identity:** Rebrand to **"SEC-Insight AI — Enterprise Document Copilot"**

2 🚨 How to Clone It (Do NOT Use git clone)

1. Open the GitHub repository link, click the green <> **Code** button, and select **Download ZIP**.
2. Extract the ZIP folder on your computer.
3. Open GitHub in your browser and create a **brand-new, empty repository** named sec-insight-ai.
4. Create a fresh folder on your laptop named sec-insight-ai and open it in VS Code.
5. Copy files over day-by-day using the plan below to establish your own clean Git commit history.

3 📅 5-Day Day-by-Day Implementation & Customization Plan

4 📌 Day 1: Project Setup & Package Configuration

Goal: Initialize the project environment and set custom package metadata.

- **Minutest Changes to Make:**

- Open package.json. Change "name": "pdf_rag" (or similar) to "name": "sec-insight-ai".
- Update "description" to "Enterprise-grade RAG engine for interactive document intelligence using Node.js and Gemini API."
- Add your name under the "author" field.
- In .env.example, change variable names (e.g., change GEMINI_KEY to GEMINI_API_KEY_SEC_INSIGHT).

- **Git Commands:**

Bash

git init

git branch -M main

git remote add origin
https://github.com/YOUR_GITHUB_USERNAME/sec-insight-ai.git

git add .

git commit -m "feat: initialize sec-insight workspace setup and npm environment configurations"

git push -u origin main

5 📌 Day 2: Ingestion Engine & Text Chunking Customization

Goal: Adjust the server-side PDF parsing and vector index configurations.

- **Minutest Changes to Make:**

- Locate the document processing script (e.g., controllers/pdfController.js or services/ingest.js).
- Modify the chunking parameters: change chunkSize from 1000 to 850 and chunkOverlap from 200 to 150.
- Change log messages in the terminal output:
 - Replace console.log("PDF uploaded") with console.log(" ⚡ SEC-Insight Ingestion: Parsing document vector stream...").

- **Git Commands:**

Bash

```
git add .
```

```
git commit -m "refactor: optimize PDF chunking strategy and document parsing middleware"
```

```
git push origin main
```

6 📌 Day 3: RAG Prompt Engineering & Gemini API Call

Goal: Customize the system prompt and response formatting logic.

- **Minutest Changes to Make:**

- Open the file that handles the Google Gemini API call (e.g., services/geminiService.js or api/chat.js).
- Update the system prompt string to:

"You are SEC-Insight AI, an expert financial and technical document analyst. Answer the user query strictly based on the provided document context below. If context is missing, output: 'Information not present in uploaded file.'"

- Set the generation parameter temperature: 0.2 (making answers more factual and precise).

- **Git Commands:**

Bash

git add .

git commit -m "feat: implement customized system prompts and low-latency Gemini API chains"

git push origin main

7 **Day 4: UI Transformation (Dark Terminal Blueprint)**

Goal: Give the interface a distinct custom look so it doesn't resemble a template.

- **Minutest Changes to Make:**

- Open public/index.html (or src/index.html). Change <title> to <title>SEC-Insight AI | Enterprise Document Intelligence</title>.
- Open your CSS file (styles.css or tailwind.config.js). Change background colors: replace standard blues with **Slate Dark (#0f172a)** and **Emerald Accents (#10b981)**.
- Update page titles in the header: rename from *"PDF Chat"* to *"SEC-Insight Workspace: Semantic Search & Q&A Panel"*.

- **Git Commands:**

Bash

git add .

git commit -m "style: apply dark blueprint theme and interactive document preview panel"

git push origin main

8 📌 Day 5: Deployment & Documentation

Goal: Deploy live on Vercel/Render and write clean documentation.

- **Minutest Changes to Make:**

- Deploy the application to Vercel or Render.
- Create a clean README.md file in the root folder:

Markdown

SEC-Insight AI

An enterprise Retrieval-Augmented Generation (RAG) platform that enables natural language Q&A across PDF documents using Node.js and Google Gemini API.

Live Demo

[Click Here to Access SEC-Insight AI](YOUR_LIVE_DEPLOYMENT_URL)

Tech Stack

- **Frontend:** HTML5, CSS3, JavaScript (Fetch API)
- **Backend:** Node.js, Express.js
- **AI Engine:** Google Gemini API (`@google/genai`)
- **Vector Search / Storage:** Pinecone / Supabase / Qdrant
- **Document Parser:** `pdf-parse`

Features

- Multi-page PDF text extraction and chunking
- Contextual vector search for cited answer generation
- Low-latency chat interface with error handling

- **Git Commands:**

Bash

```
git add .
```

```
git commit -m "docs: finalize deployment setup and write comprehensive  
README documentation"
```

```
git push origin main
```

9 🚨 **Crucial Pre-Submission Check**

Before sharing your repository, press Ctrl + Shift + F in VS Code and search across the entire project for original names (like Aasuyadav1, NitinPandit, or pdf_rag). Replace every single instance to ensure the project looks entirely original!

Project Title:

SEC-Insight AI — Enterprise GenAI Document Intelligence Copilot

Tech Stack & Developer Tools:

Full-Stack & Backend: React.js, Node.js, Express.js, JavaScript (ES6+), HTML5/CSS3

AI & RAG Engine: @google/genai (Gemini API SDK), pdf-parse, Vector Storage (Pinecone / Supabase pgvector), REST APIs

Tooling & Cloud: Git/GitHub, Postman, Vercel (Frontend), Render (Backend)

Resume Bullet Points (Human-Written & ATS-Optimized):

- **Engineered a Retrieval-Augmented Generation (RAG) platform** using React and Node.js to enable real-time, natural language Q&A across lengthy enterprise PDF documents.
- **Built a server-side document ingestion engine** with pdf-parse to extract text from multi-page files, chunk content semantically, and manage vector embedding indices.
- **Integrated Google Gemini API via JavaScript SDK** to execute context-aware prompt chains, delivering fast, cited answer summaries while preventing model hallucinations.
- **Designed a split-screen workspace interface** featuring an interactive document viewer alongside a chat workspace for instant side-by-side source verification.

pdf-parse, pgvector, and Vercel are three critical building blocks of your AI Copilot full-stack architecture.

10 1. pdf-parse (The Document Translator)

11 What is it?

When an end-user drops a PDF document (like a 50-page annual financial report) into your app, Node.js cannot read or understand standard PDF visual data out of the box. pdf-parse is an open-source JavaScript library (an npm package) designed for Node.js backend servers.

12 What does it do in your project?

- **Extracts Plain Text:** It opens the binary structure of a .pdf file and strips out all formatting, rendering it as raw, plain string text.
- **Prepares Content for AI:** After extracting the raw text, your Node.js code can slice that text into small readable paragraphs (e.g., 500-character chunks) to send over to the Google Gemini API.

Analogy: Think of pdf-parse as a translator converting a scanned document into a plain text file so an assistant can read it word-for-word.

13 2. pgvector (The AI Vector Database Engine)

14 What is it?

pgvector is an open-source extension for **PostgreSQL** (the world's most popular relational database). It is heavily used in cloud database platforms like **Supabase**.

Standard databases only search for *exact word matches* (e.g., searching "automobile" will fail to find "car"). pgvector lets your database store and search **Vector Embeddings**—long lists of numerical coordinates representing the deep *semantic meaning* of text.

15 What does it do in your project?

- **Stores Document Coordinates:** After pdf-parse extracts text and Gemini converts that text into numerical vectors, pgvector saves those vector coordinates in your database alongside your text.
- **Runs Similarity Searches:** When a user asks "*What were the company's risks in 2025?*", pgvector calculates the distance between the question's vector and the document's vectors. It instantly pulls up the top 3 passages with identical *meanings*, even if they use completely different words!

Analogy: Imagine a huge library where books aren't sorted alphabetically by author, but by *topic similarity*. pgvector is the GPS map that instantly takes you to the shelf containing the exact idea you are looking for.

16 3. Vercel (The Deployment Cloud Platform)

17 What is it?

Vercel is a modern cloud deployment platform created by the team behind Next.js. It hosts web applications on global edge servers so users anywhere in the world can load your project with near-zero latency.

18 What does it do in your project?

- **Hosts Your Live App:** Instead of your app running locally on localhost:3000 on your personal laptop, Vercel gives you a live, production URL (e.g., <https://sec-insight-ai.vercel.app>) that recruiters can click on and test live from their phones or laptops.

- **Automated Git CI/CD:** Every time you run `git push origin main` in VS Code, Vercel automatically catches the code update, builds the project in the cloud, and updates your live site within seconds.

Analogy: If your local machine is your kitchen where you cook the meal (write your code), Vercel is the restaurant storefront that opens the doors and serves the dish to real customers across the globe.

19 🧠 How They Work Together in Your AI Copilot

[User Uploads PDF]

|



1. ``pdf-parse`` extracts raw text from the file

|



2. Gemini API converts text chunks into mathematical vectors

|



3. ``pgvector`` stores vectors in Supabase & handles similarity search

|



4. Vercel hosts the entire live React UI and Serverless functions for users

20 What is SEC-Insight AI?

SEC-Insight AI is a **GenAI-powered Document Intelligence Copilot**.

It is designed to solve a common problem for financial analysts, developers, and researchers: reading through lengthy enterprise files (such as 100-page financial filings, annual 10-K/10-Q SEC reports, or complex technical PDFs) is slow and tedious.

SEC-Insight AI uses **Retrieval-Augmented Generation (RAG)** to parse PDF documents, index them, and answer natural language questions in real time with exact inline citations pointing back to the original text.

21 How to Use the App (Step-by-Step Workflow)

Using the platform involves four main steps:

22 1. Document Upload & Parsing

- Open the web interface on your browser.
- Drag and drop a multi-page PDF document (e.g., an annual corporate report, a technical manual, or a financial audit).
- **Behind the Scenes:** The backend runs a parser (like pdf-parse in Node.js) to strip raw text out of the PDF and split it into small semantic chunks.

23 2. Automatic Embeddings & Indexing

- Once the text is extracted, the application sends the text chunks to an AI model (like the **Google Gemini API**) to convert them into mathematical vectors.
- The vectors are saved in a vector database (like **Pinecone** or Supabase pgvector) so the system can understand the contextual *meaning* of the content.

24 3. Natural Language Querying

- Type any question into the interactive chat window (e.g., *"What were the major security risks mentioned on page 12?"* or *"Summarize the Q3 revenue metrics"*).
- The application performs a similarity search across your vector database to pull out only the relevant text passages matching your prompt.

25 4. Side-by-Side Review & Verification

- The AI engine streams a precise, context-aware answer into your chat workspace.
- You can click on the **interactive citations** next to the answer to highlight and verify the exact source sentence on the original PDF side-by-side.

The complete workflow and data flow of **SEC-Insight AI** breaks down into two main phases: **Document Ingestion** (processing the file) and **Query Execution** (asking a question and receiving an answer).

C# Corner

26 Phase 1: Document Upload & Ingestion Pipeline

When you drag and drop a PDF into the application, it moves through the system step-by-step:

[User UI] —► [Express.js Backend] —► [pdf-parse Engine] —► [Gemini Vector API] —► [Supabase (pgvector)]

1. User Action (React.js / Tailwind CSS):

You select a 50-page PDF report and click **Upload**. The React UI sends the raw PDF file over an HTTP POST request to your backend server.

2. Backend Server (Node.js & Express.js):

Your Express server intercepts the incoming request and temporarily holds the file in memory.

3. Text Extraction (pdf-parse):

Node.js passes the binary PDF file to the pdf-parse library. It strips out formatting, images, and layout elements, returning a single, clean text string of all the words in the document.

C# Corner

4. Semantic Text Chunking (JavaScript):

A 50-page string is too large to process all at once. Your Node.js script automatically slices the long text string into smaller overlapping paragraphs (e.g., 850-character chunks with a 150-character overlap).

DEV Community

5. Generating Vector Embeddings (@google/genai SDK):

Node.js sends these text chunks to the **Google Gemini API**. Gemini converts each text chunk into a "Vector Embedding"—a list of numerical coordinates representing the deep contextual *meaning* of the text.

IBM

6. Vector Storage (Supabase pgvector / Pinecone):

Your server takes those numerical vectors—along with metadata like the original text string and page numbers—and stores them inside **Supabase (pgvector)**.

DEV Community

27 Phase 2: Natural Language Querying & Answer Generation (RAG)

When you ask a question about the document, the application runs a **Retrieval-Augmented Generation (RAG)** search:

[User Query] —► [Gemini Vector API] —► [Vector Similarity Search] —► [Gemini Prompt API] —► [React UI]

1. User Query Input:

You type a question into the chat workspace (e.g., *"What were the financial risks listed on page 10?"*).

2. Question Vectorization:

Your Node.js backend sends your question to the **Google Gemini API** to turn your prompt into a mathematical vector array.

IBM

3. Context Retrieval (Supabase pgvector Similarity Search):

pgvector performs a mathematical cosine similarity calculation, matching your question's vector against all stored document vectors. It instantly fetches the **top 3 most relevant text chunks** from the database.

C# Corner+ 1

4. Context-Aware Prompt Generation:

Node.js constructs a prompt that combines the retrieved document text with your original question:

"Using ONLY the context passages below, answer the user's question. Context: [Top 3 Retrieved Text Chunks]. Question: What were the financial risks listed on page 10?"

5. AI Answer Streaming (@google/genai API):

Node.js sends this engineered prompt to the **Google Gemini API**. Gemini reads *only* the provided document passages and outputs a precise, cited answer without hallucinating.

SingleStore

6. UI Rendering (React.js & Vercel):

The Node.js server streams the generated text response to the React frontend hosted on **Vercel**. The UI displays the AI answer alongside clickable page citations next to the original PDF preview window.

C# Corner+ 1

28 Summary of How Every Tool Performs Its Function

- **React.js & Tailwind CSS:** Renders the split-screen web page, chat interface, dark theme styling, and buttons.
- **Node.js & Express.js:** Functions as the central backend orchestrator that moves data between the browser, file parsers, database, and AI services.

C# Corner

- **pdf-parse:** Converts raw .pdf files into readable plain-text strings.

C# Corner

- **Google Gemini API (@google/genai):** Converts text into mathematical vectors (embeddings) and generates contextual answers to user questions.

IBM

- **Supabase (pgvector):** The relational database extension storing text chunks and searching numerical vectors to find contextual matches.

DEV Community

- **Vercel & Render:** The cloud hosting infrastructure that runs your React frontend (Vercel) and Express backend API (Render) live on the internet.

Here is a comprehensive bank of **32 technical, architectural, and behavioral interview questions with full, expert-level answers** specifically tailored to your **SEC-Insight AI (GenAI Document Copilot)** project built with **React, Node.js, pdf-parse, Google Gemini API, and Supabase (pgvector)**.

-  **Category 1: High-Level RAG Architecture & System Design**
- **Q1. What is RAG, and why did you build a RAG-based Document Copilot instead of fine-tuning an LLM?**
- **Answer: Retrieval-Augmented Generation (RAG)** is an architecture where external knowledge is fetched at runtime from a database and injected as context into the LLM prompt before generating an answer. Fine-tuning is expensive, slow, modifies model weights permanent-state, and suffers from memory cutoffs. I chose RAG because:

1. It handles **dynamic multi-page enterprise PDFs** uploaded on demand without needing retraining.
 2. It keeps company data private.
 3. It dramatically reduces **hallucinations** by forcing the LLM to ground its response strictly in retrieved text passages.
- **Q2. Walk me through the end-to-end data flow when a user uploads a PDF and asks a question.**
 - **Answer:**
 1. **Upload & Parse:** The React UI sends the binary .pdf via REST API to Express.js. Express passes it to pdf-parse to strip out raw text strings.
 2. **Chunking:** Node.js splits the raw text into overlapping paragraphs (850 characters with a 150-character overlap).
 3. **Embedding:** Node.js sends the text chunks to the Google Gemini Embeddings API via @google/genai to generate 768-dimensional (or 1536-dimensional) vector representations.
 4. **Store:** Text chunks and their corresponding vectors are stored in Supabase using the pgvector extension.
 5. **Query:** When the user asks a question, Express generates a vector embedding for the prompt, runs a cosine similarity search against pgvector, retrieves the top 3 matching chunks,

and injects them into a system prompt for Gemini to stream the answer back to the React UI.

- **Q3. What is the difference between Dense Retrieval and Sparse Retrieval? Which does your app use?**

- **Answer:**

- **Sparse Retrieval (e.g., BM25 / SQL LIKE):** Matches exact keywords and term frequencies. It fails if the query uses synonyms (e.g., "car" vs. "automobile").
- **Dense Retrieval (Vector Search):** Transforms text into mathematical vectors to search by semantic meaning, intent, and context.
- **Our App:** Uses **dense retrieval** via pgvector cosine similarity so users can ask questions using natural language rather than exact PDF search keywords.

- **Q4. How do you prevent multi-tenant data leaks in pgvector?**

- **Answer:** Each uploaded document chunk saved in Supabase includes metadata columns: `tenant_id`, `user_id`, and `document_id`. When executing vector similarity searches, we apply **Metadata Filtering**:

SQL

```
SELECT content FROM document_embeddings
```

WHERE tenant_id = req.user.tenantId

ORDER BY embedding <=> query_embedding LIMIT 3;

This ensures that vector similarity queries are strictly scoped to the authenticated user's organization.

- **Q5. What is "Top-K" retrieval, and how did you select your value?**

- **Answer: Top-K** represents the maximum number of vector matches fetched from the database to inject into the LLM prompt. We selected **Top-3 (K=3)** because:

- Setting K too high (e.g., K=20) causes **"Lost in the Middle" syndrome** (where the LLM ignores context in long prompts) and increases API token costs.
- Setting K too low (e.g., K=1) risks missing necessary supporting context spread across multiple pages.

-  **Category 2: Ingestion, Parsing & Vector Database (pdf-parse & pgvector)**

- **Q6. What is pdf-parse, and why is it necessary?**

- **Answer:** Node.js cannot read binary .pdf structures natively. pdf-parse is a server-side library that extracts raw ASCII/UTF-8 text content, page numbers, and basic text metadata from PDF file buffers so our Node.js pipeline can slice it into readable text chunks.

- **Q7. How does chunking work, and why not pass an entire 50-page PDF directly to Gemini?**

- **Answer:** Chunking breaks long documents into smaller segments. Passing an entire 50-page document:

1. Exceeds prompt token limits or incurs high API costs.
2. Dilutes vector specificity (an embedding of an entire book loses precise sentence meanings).
3. Increases response latency significantly.

- **Q8. What chunk size and overlap strategy did you use, and why?**

- **Answer:** We used **Recursive Text Chunking** with **850 characters per chunk** and **150 characters overlap**.

- The **overlap** ensures that sentences or facts cut off at chunk boundaries are not lost out of context.
- The size strikes a balance between keeping enough context for Gemini to understand and maintaining sharp vector retrieval precision.

- **Q9. What is a vector embedding? Explain it simply.**

- **Answer:** A vector embedding is a list of floating-point numbers (e.g., [0.012, -0.453, 0.891...]) that plots the conceptual meaning of a piece of text as a coordinate point in multi-dimensional space.

Words or sentences with similar meanings (like "profit" and "revenue") sit close to each other in this mathematical space.

- **Q10. What is pgvector, and how does Cosine Similarity search work?**

- **Answer:** pgvector is an extension that enables PostgreSQL to store vectors in specialized columns and run vector indexes (like HNSW or IVFFlat). Cosine similarity calculates the cosine of the angle between two vectors in multi-dimensional space:

- Angle close to 0° (Cosine score ~ 1.0): Texts have nearly identical meanings.
- Angle of 90° (Cosine score ~ 0.0): Texts are unrelated.

- **Q11. How do you handle scanned PDFs containing images instead of selectable text?**

- **Answer:** Standard text parsers like pdf-parse return empty strings for image-based PDFs. In our production pipeline design, if pdf-parse returns fewer than 50 characters for a page, the file falls back to an **Optical Character Recognition (OCR)** engine (like Tesseract.js or Google Vision API) to extract text from images before chunking.

- 🤖 **Category 3: Gemini API & Prompt Engineering**

- **Q12. How do you structure the system prompt to stop the Gemini API from hallucinating?**

- **Answer:** We use strict **System Prompt Grounding**:

"You are SEC-Insight AI. Answer the user question strictly based on the provided document context below. If the answer cannot be found in the context, explicitly respond: 'Information not present in uploaded document.' Do not use external knowledge."

- **Q13. What temperature setting did you choose for your Gemini LLM API calls, and why?**

- **Answer:** We set temperature: 0.2. Temperature controls response randomness (0.0 = completely deterministic/factual; 1.0 = highly creative). A low temperature ensures factual accuracy for financial and enterprise document Q&A.

- **Q14. What is the difference between @google/genai (SDK) and making raw REST API fetch requests?**

- **Answer:** The official @google/genai Node.js SDK provides native TypeScript types, automatic retry handling for rate limits (HTTP 429), built-in stream handling for token responses, and simple configuration for model parameters compared to manual fetch() calls.

- **Q15. How do you generate source citations in the response?**

- **Answer:** When storing text chunks in Supabase, we retain metadata: { pageNumber: 14, chunkId: 102 }. When building the context string for Gemini, we format it as:

[Page 14]: "In Q3, total operational costs rose by 12%..."

We instruct the prompt: *"Include the inline [Page X] tag whenever referencing a fact."*

- ⚡ **Category 4: Node.js, Express & JavaScript Technicals**
- **Q16. Why did you choose Node.js over Python for building this RAG application?**
- **Answer:** Using Node.js allowed us to maintain a unified full-stack JavaScript/TypeScript codebase. Express.js efficiently handles asynchronous I/O operations (like streaming API calls to Gemini and database queries to Supabase) using an event-driven model without blocking the main execution thread.
- **Q17. Explain the Node.js Event Loop and non-blocking I/O during heavy PDF uploads.**
- **Answer:** Node.js runs on a single-threaded Event Loop. Network requests and disk writes are delegated to Libuv thread pools. When parsing large files, async operations ensure the server remains responsive to incoming HTTP requests while background tasks process text chunks.
- **Q18. How do you prevent memory leaks when handling file uploads in Express?**
- **Answer:** Instead of keeping whole files in server RAM, we use stream buffers via multer memory storage with strict byte limits (limits: { fileSize: 10 * 1024 * 1024 }), freeing up memory once pdf-parse finishes text extraction.

- **Q19. What is Express middleware, and how did you use it here?**

- **Answer:** Middleware functions execute sequentially before reaching final route handlers. In this project, we used:

1. **Authentication Middleware:** Validates JWT access tokens.
2. **File Validation Middleware:** Checks file MIME types (application/pdf).
3. **Global Error Middleware:** Captures unhandled exceptions and returns structured JSON error responses.

- **Q20. What is the difference between Promise.all and sequential await when embedding text chunks?**

- **Answer:**

- **Sequential await:** Waits for chunk 1 to embed before starting chunk 2 (slow, blocking the network pipeline).
- **Promise.all:** Sends multiple embedding API calls in parallel, resolving when all vectors return. We batch promises (e.g., 5 at a time) to stay within Gemini API rate limits.

- 🎨 **Category 5: React.js UI & Frontend Execution**

- **Q21. How is the split-screen PDF preview and chat window coordinated in React?**

- **Answer:** We use a centralized parent React state holding `activePage` and `selectedDocument`. Clicking a citation tag (e.g., [Page 4]) inside the chat component triggers a state update that scrolls the embedded PDF viewer component directly to page 4.
- **Q22. How do you achieve real-time text streaming (chat typing effect) in React?**
- **Answer:** We use **Server-Sent Events (SSE)** or readable fetch streams from our Node.js backend (`res.setHeader('Content-Type', 'text/event-stream')`). The React client reads the stream chunks using `ReadableStreamReader` and appends incoming text tokens to the UI state progressively.
- **Q23. What is the difference between `useState` and `useRef` when handling chat inputs?**
- **Answer:** `useState` triggers component re-renders on state change (ideal for chat message arrays and loading spinners). `useRef` persists mutable values without triggering re-renders (ideal for storing DOM node references like auto-scrolling the chat container to the bottom).
- **Q24. How do you optimize React performance when rendering hundreds of chat messages?**
- **Answer:** We use component memoization (`React.memo`) on individual message bubbles and component virtualizers (like `react-window`) so only visible messages are rendered in the DOM.

- 🧠 **Category 6: Deployment, Security & Production Challenges**
- **Q25. Where is your application hosted, and how is the pipeline connected?**
- **Answer:**
 - **Frontend:** Deployed on **Vercel** for fast edge distribution.
 - **Backend:** Hosted on **Render** (Node.js web service).
 - **Database:** **Supabase Cloud** (Managed PostgreSQL + pgvector).
 - **CI/CD:** Automatic deployments trigger on pushing code to the main branch on GitHub.
- **Q26. How do you securely store and manage your Gemini API key and Supabase credentials?**
- **Answer:** Environment secrets are stored in .env files locally (added to .gitignore) and injected via deployment configuration settings on Render and Vercel. API keys are never exposed to the client-side React code.
- **Q27. What happens if the Gemini API goes down or hits rate limits (HTTP 429)?**
- **Answer:** We implement an exponential backoff retry mechanism inside our backend API handler. If rate limits persist, the system falls

back to a secondary model endpoint or returns a clean fallback error to the frontend.

- **Q28. How would you scale this app to handle millions of documents?**

- **Answer:**

1. **Async Job Queues:** Offload parsing/chunking to background worker pools (using Redis and BullMQ).
2. **Dedicated Vector DB Indexing:** Add **HNSW (Hierarchical Navigable Small World)** vector indexing in pgvector for fast approximate nearest-neighbor searches.
3. **Caching Layer:** Cache frequent query vectors and responses in Redis.

-  **Category 7: Behavioral & Problem-Solving Scenarios**

- **Q29. Tell me about a difficult technical challenge you faced while building this project.**

- **Answer:** *"Initially, retrieved context chunks were cutting off mid-sentence, causing Gemini to hallucinate missing information. I solved this by transitioning from simple fixed-length character splitting to a **Recursive Character Splitting** approach with a 150-character overlap. This kept full sentences intact across chunk boundaries, increasing retrieval relevance."*

- **Q30. Why did you choose Supabase (pgvector) instead of dedicated vector databases like Pinecone?**
 - **Answer:** *"Pinecone is great, but Supabase allowed us to keep traditional user data (users, auth logs, metadata) and vector embeddings in a single unified PostgreSQL database. This eliminated the need to synchronize data across multiple separate services."*
- **Q31. If you had 2 more weeks to work on this project, what feature would you add next?**
 - **Answer:** *"I would implement **Hybrid Search** (combining BM25 keyword matching with vector search) and a **Re-ranking pipeline** (using a cross-encoder model) to further improve search accuracy for technical jargon and financial acronyms."*
- **Q32. How do you measure if the RAG answers are accurate?**
 - **Answer:** *"We evaluate using **Faithfulness** (checking if generated claims exist in the retrieved text) and **Answer Relevance** (checking if the generated response directly answers the query). Prompting Gemini to provide source page citations makes it easy for users to manually verify answers."*